AI VIET NAM
@aivietnam.edu.vn

AI VIET NAM – AI COURSE 2025

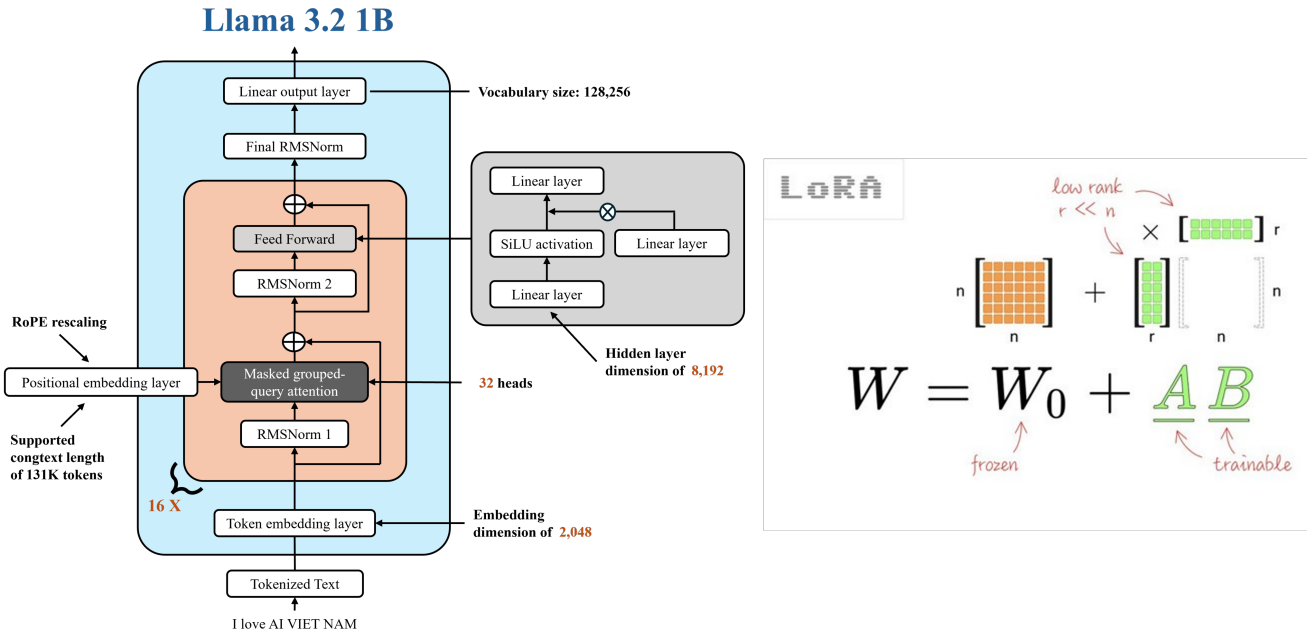
LLMs Fine-tuning

Nguyễn Quốc Thái Hồ Quang Hiển Đinh Quang Vinh

I. Giới thiệu

Trong những năm gần đây, fine-tuning mô hình ngôn ngữ lớn theo hướng tiết kiệm tài nguyên đã trở thành một hướng rất quan trọng trong cả nghiên cứu lẫn ứng dụng thực tế. Thay vì cập nhật toàn bộ tham số của mô hình với chi phí lớn, nhiều phương pháp hiện đại chỉ tinh chỉnh một phần nhỏ tham số nhưng vẫn giữ được chất lượng đầu ra tốt. Trên nền tảng đó, các kỹ thuật như SFT, LoRA và QLoRA ngày càng được sử dụng rộng rãi trong quá trình tùy biến LLM cho các mục tiêu cụ thể.

Hình dưới đây minh họa hai thành phần trung tâm trong tài liệu. Bên trái là kiến trúc tổng quát của `Llama-3.2-1B`, còn bên phải là nguyên lý của `LoRA`, trong đó trọng số gốc được giữ cố định và chỉ học thêm một cập nhật hạng thấp. Cách trình bày này giúp người học dễ hình dung mối liên hệ giữa mô hình nền và cơ chế fine-tuning hiệu quả tham số.



Hình 1: Minh họa kiến trúc `Llama-3.2-1B` và nguyên lý cập nhật tham số hạng thấp trong `LoRA`.

Tài liệu này tập trung vào quá trình tinh chỉnh mô hình `Llama-3.2-1B-Instruct`, một mô hình ngôn ngữ gọn nhẹ, phù hợp cho thực nghiệm khi tài nguyên phần cứng còn hạn chế. Đây cũng là lựa chọn phù hợp để quan sát sự khác biệt giữa fine-tuning toàn phần và fine-tuning hiệu quả tham số.

Về mặt kỹ thuật, nội dung bắt đầu từ SFT như bước nền tảng để mô hình học cách phản hồi theo dữ liệu có giám sát, sau đó mở rộng sang LoRA và QLoRA nhằm giảm chi phí huấn luyện. LoRA bổ sung các ma trận hạng thấp vào một số lớp của mô hình để chỉ cập nhật một lượng nhỏ tham số mới, trong khi QLoRA tiếp tục tối ưu bộ nhớ bằng cách lượng tử hóa mô hình gốc và chỉ huấn luyện các adapter cần thiết.

Bên cạnh mô hình và phương pháp, tài liệu cũng giới thiệu các thư viện quan trọng như `TRL`, `PEFT` và `Unsloth`. Những công cụ này hỗ trợ xây dựng pipeline huấn luyện rõ ràng, tiết kiệm bộ nhớ và thuận tiện hơn trong thực nghiệm. Mục tiêu của tài liệu là giúp người học hiểu được mối liên hệ giữa mô hình nền, chiến lược fine-tuning và hệ sinh thái thư viện hỗ trợ để có thể xây dựng một quy trình tinh chỉnh LLM hiệu quả trong thực tế.

Mục lục

I. Giới thiệu **1**

II. Tổng quan về Fine-Tuning LLM **4**

III. Giới thiệu `Llama-3.2-1B-Instruct` **4**

IV. Supervised Fine-Tuning (SFT) **5**

V. LoRA và QLoRA **6**

V.1. LoRA 6

V.2. QLoRA 7

VI. Các thư viện: `TRL`, `PEFT`, `Unsloth` **8**

VII. Thực hành SFT với Llama-3.2-1B-Instruct bằng `TRL` **9**

VIII. Thực hành SFT với Llama-3.2-1B-Instruct bằng `Unsloth` **19**

IX. Câu hỏi trắc nghiệm **28**

Phụ lục **31**

II. Tổng quan về Fine-Tuning LLM

Fine-tuning là quá trình tiếp tục huấn luyện một mô hình ngôn ngữ đã được tiền huấn luyện sẵn để mô hình thích nghi tốt hơn với một nhiệm vụ, một miền dữ liệu, hoặc một phong cách phản hồi cụ thể. Thay vì xây dựng mô hình từ đầu, cách làm này tận dụng tri thức đã có của mô hình nền, từ đó rút ngắn thời gian huấn luyện và giảm đáng kể chi phí tính toán.

Trong thực tế, không phải lúc nào cũng cần cập nhật toàn bộ tham số của mô hình. Với các mô hình lớn, full fine-tuning thường đòi hỏi rất nhiều bộ nhớ GPU, thời gian huấn luyện dài, và chi phí lưu trữ cao khi phải giữ riêng một bản mô hình đầy đủ cho từng tác vụ. Vì vậy, các hướng fine-tuning hiệu quả tham số ngày càng trở nên quan trọng, đặc biệt trong môi trường học tập, nghiên cứu, và triển khai thực tế với tài nguyên hạn chế.

Một cách nhìn trực quan là xem mô hình nền như một người đã có kiến thức ngôn ngữ rất tốt, còn fine-tuning là bước huấn luyện thêm để người đó trả lời đúng hơn theo yêu cầu mới. Nếu chỉnh sửa toàn bộ mô hình, ta tác động lên toàn bộ hệ thống tri thức. Nếu chỉ dùng các phương pháp như LoRA hoặc QLoRA, ta giữ nguyên phần lớn mô hình gốc và chỉ học thêm một phần nhỏ để điều chỉnh hành vi đầu ra.

Trong tài liệu này, quá trình fine-tuning được xây dựng xoay quanh ba thành phần chính:

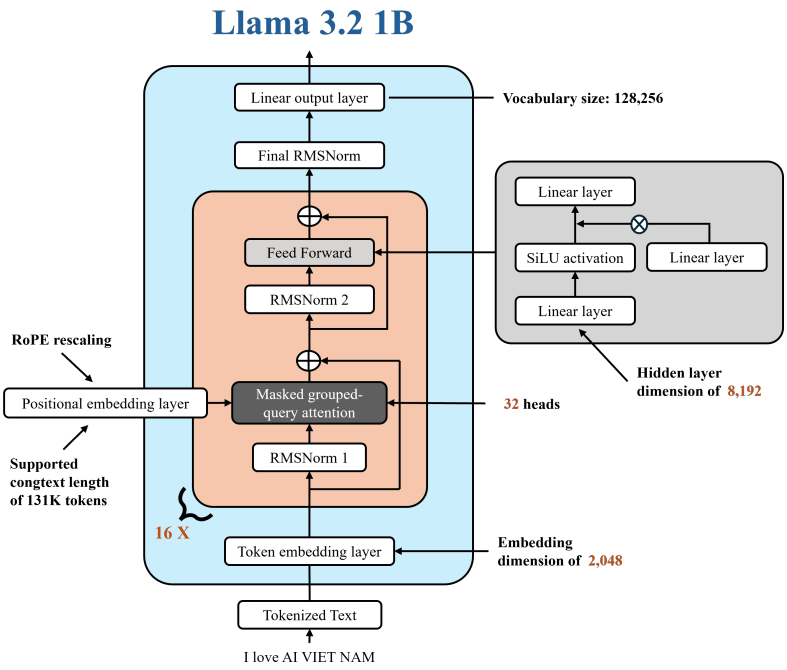
- Supervised Fine-Tuning (SFT) để mô hình học từ dữ liệu có giám sát.
- LoRA để giảm số lượng tham số cần huấn luyện.
- QLoRA để tiếp tục giảm yêu cầu bộ nhớ khi huấn luyện.

Mục tiêu chung là xây dựng một pipeline tinh chỉnh LLM vừa rõ ràng về mặt khái niệm, vừa thực tế về mặt triển khai. Người học không chỉ biết fine-tuning là gì, mà còn hiểu vì sao cần các kỹ thuật tiết kiệm tài nguyên và chúng liên kết với nhau như thế nào trong một quy trình hoàn chỉnh.

III. Giới thiệu `Llama-3.2-1B-Instruct`

Mô hình được sử dụng trong tài liệu là `Llama-3.2-1B-Instruct`. Đây là một mô hình ngôn ngữ thuộc họ Llama, có quy mô tương đối gọn nhẹ, phù hợp cho các thí nghiệm fine-tuning trong bối cảnh phần cứng chưa quá mạnh. Việc chọn phiên bản 1B tham số giúp người học dễ tiếp cận hơn so với các mô hình lớn hơn, đồng thời vẫn đủ để quan sát các cơ chế huấn luyện và thích nghi của LLM.

Tên gọi của mô hình cũng gợi ý rõ vai trò của nó. Phần `1B` cho biết mô hình có khoảng một tỷ tham số. Phần `Instruct` cho biết mô hình đã được tinh chỉnh theo hướng làm theo chỉ dẫn, nghĩa là đầu ra của nó thường phù hợp hơn với các prompt dạng yêu cầu, câu hỏi, hoặc nhiệm vụ được diễn đạt bằng ngôn ngữ tự nhiên.



Hình 2: Kiến trúc tổng quan của mô hình `Llama-3.2-1B`.

Có thể phân biệt ngắn gọn giữa base model và instruct model như sau:

- Base model mạnh về biểu diễn ngôn ngữ, nhưng chưa tối ưu để trả lời theo chỉ dẫn của người dùng.
- Instruct model đã được huấn luyện thêm để phản hồi đúng định dạng, đúng vai trò, và đúng mục tiêu hơn trong các tình huống hỏi đáp hoặc hỗ trợ tác vụ.

Ví dụ, khi đưa vào một yêu cầu như `Hãy giải thích khái niệm overfitting bằng ngôn ngữ đơn giản`, instruct model thường có xu hướng trả lời trực tiếp, rõ ràng, và bám sát yêu cầu hơn. Đây là một đặc điểm rất quan trọng khi xây dựng các ứng dụng hội thoại, trợ lý ảo, hoặc hệ thống sinh nội dung có kiểm soát.

Việc chọn `Llama-3.2-1B-Instruct` trong bài này còn có một lợi thế khác: nó cho phép người học tập trung vào bản chất của pipeline fine-tuning thay vì bị choáng ngợp bởi hạ tầng quá phức tạp. Khi mô hình đủ nhỏ để thực nghiệm, các bước như chuẩn bị dữ liệu, thêm adapter, cấu hình huấn luyện và đánh giá đầu ra sẽ trở nên dễ quan sát và dễ phân tích hơn.

IV. Supervised Fine-Tuning (SFT)

Supervised Fine-Tuning, thường được viết tắt là SFT, là bước nền tảng trong nhiều pipeline tinh chỉnh LLM. Ý tưởng của SFT là sử dụng dữ liệu có giám sát để dạy mô hình cách phản hồi theo một định dạng hoặc phong cách mong muốn. Nói cách khác, mô hình được nhìn thấy các cặp

đầu vào và đầu ra mẫu, sau đó học cách sinh ra phản hồi phù hợp khi gặp những yêu cầu tương tự.

Trong ngữ cảnh instruction tuning, dữ liệu SFT thường được tổ chức quanh các thành phần quen thuộc sau:

- Instruction: Yêu cầu hoặc nhiệm vụ mà người dùng đưa ra.
- Input: Ngữ cảnh bổ sung, nếu bài toán cần thêm thông tin.
- Response: Câu trả lời mẫu mà mô hình cần học theo.

Một ví dụ đơn giản có thể được viết như sau:

```
Instruction: Hãy tóm tắt đoạn văn sau trong 2 câu.
Input: Trí tuệ nhân tạo đang được ứng dụng trong giáo dục...
Response: Trí tuệ nhân tạo đang hỗ trợ nhiều hoạt động giáo dục...
```

Khi huấn luyện, mô hình sẽ học cách ánh xạ từ phần prompt đầu vào sang phần phản hồi mong muốn. Nếu dữ liệu được tổ chức tốt, mô hình dần hình thành thói quen trả lời đúng vai trò, đúng cấu trúc và đúng kiểu nội dung mà bài toán yêu cầu. Đây là lý do vì sao chất lượng dữ liệu SFT thường ảnh hưởng rất mạnh đến chất lượng mô hình sau fine-tuning.

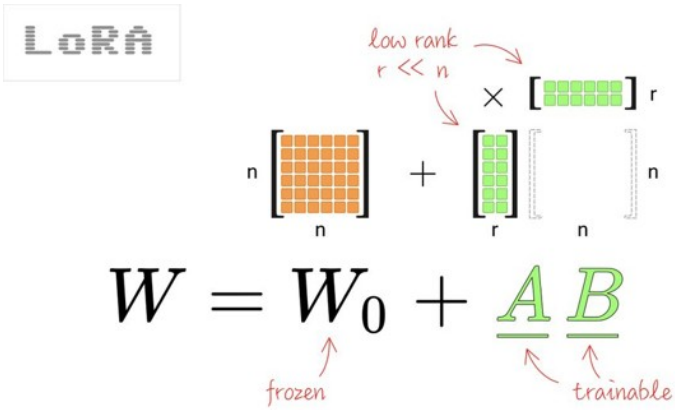
Một điểm quan trọng là SFT không chỉ dạy mô hình "trả lời cái gì"mà còn dạy mô hình "trả lời như thế nào". Chẳng hạn, cùng một câu hỏi nhưng ta có thể muốn mô hình trả lời ngắn gọn, mang tính học thuật, thân thiện với người mới học, hoặc theo phong cách hỗ trợ kỹ thuật. Những khác biệt đó đều có thể được phản ánh thông qua cách xây dựng bộ dữ liệu huấn luyện.

Trong pipeline của tài liệu này, SFT đóng vai trò là lớp nền về mặt mục tiêu học. Sau khi xác định rõ dữ liệu instruction-response và cách mô hình cần phản hồi, các kỹ thuật như LoRA hoặc QLoRA sẽ được dùng để làm cho quá trình huấn luyện trở nên nhẹ hơn và khả thi hơn trên phần cứng phổ thông.

V. LoRA và QLoRA

V.1. LoRA

LoRA, viết tắt của Low-Rank Adaptation, là một phương pháp fine-tuning hiệu quả tham số. Thay vì cập nhật trực tiếp toàn bộ ma trận trọng số lớn của mô hình, LoRA giữ nguyên trọng số gốc và chỉ học thêm một phần hiệu chỉnh nhỏ có cấu trúc hạng thấp. Nhờ đó, số lượng tham số cần huấn luyện giảm đi rất nhiều so với full fine-tuning.



Hình 3: Minh họa nguyên lý `LoRA` với cập nhật trọng số hạng thấp $W = W_0 + AB$.

Ý tưởng cốt lõi của LoRA thường được biểu diễn bằng công thức:

$$W = W_0 + AB$$

Trong đó:

- W_0 là trọng số gốc của mô hình và thường được giữ cố định.
- A và B là hai ma trận nhỏ được huấn luyện thêm.
- Hạng r của phép phân rã thường nhỏ hơn rất nhiều so với kích thước đầy đủ của ma trận gốc.

Cách tiếp cận này mang lại hai lợi ích rất rõ. Thứ nhất, bộ nhớ dùng cho gradient và optimizer state giảm xuống đáng kể vì số tham số trainable ít hơn. Thứ hai, việc lưu trữ kết quả fine-tuning cũng nhẹ hơn vì nhiều trường hợp chỉ cần lưu adapter thay vì lưu cả mô hình đầy đủ.

Có thể hình dung trực giác rằng mô hình gốc giống như một hệ thống lớn đã được xây dựng hoàn chỉnh, còn LoRA chỉ gắn thêm một lớp điều chỉnh nhỏ để thay đổi hành vi ở một vài điểm quan trọng. Điều này đặc biệt hữu ích khi muốn tùy biến mô hình cho nhiều tác vụ khác nhau mà không phải sao chép toàn bộ trọng số mỗi lần.

V.2. QLoRA

QLoRA có thể được hiểu là sự kết hợp giữa LoRA và quantization. Nếu LoRA giúp giảm số tham số cần huấn luyện, thì QLoRA tiến thêm một bước bằng cách giảm bộ nhớ dùng để chứa mô hình gốc. Cụ thể, mô hình nền được lượng tử hóa xuống mức thấp hơn, thường là 4-bit, trong khi các adapter LoRA vẫn được huấn luyện để mô hình thích nghi với nhiệm vụ mới.

Nhờ cách làm này, QLoRA đặc biệt phù hợp trong các tình huống phần cứng giới hạn. Người học vẫn có thể fine-tune mô hình tương đối lớn trên GPU phổ thông mà không cần cập nhật toàn bộ mô hình ở độ chính xác cao như cách truyền thống.

Có thể tóm tắt sự khác nhau giữa ba hướng phổ biến như sau:

- Full Fine-Tuning: Cập nhật toàn bộ tham số của mô hình.
- LoRA: Giữ nguyên mô hình gốc, chỉ huấn luyện adapter hạng thấp.
- QLoRA: Giữ nguyên mô hình gốc ở dạng lượng tử hóa và huấn luyện adapter hạng thấp.

Về mặt trực giác, nếu full fine-tuning là sửa lại toàn bộ cỗ máy, thì LoRA là gắn thêm một bộ điều chỉnh nhỏ, còn QLoRA là vừa nén cỗ máy xuống gọn hơn vừa dùng bộ điều chỉnh đó để tiết kiệm tài nguyên tối đa. Chính vì vậy, LoRA và đặc biệt là QLoRA đã trở thành lựa chọn rất phổ biến trong các bài toán tinh chỉnh LLM hiện nay.

VI. Các thư viện: `TRL`, `PEFT`, `Unsloth`

Để xây dựng một pipeline fine-tuning rõ ràng và thực tế, ngoài mô hình và phương pháp huấn luyện, ta còn cần các thư viện hỗ trợ phù hợp. Trong tài liệu này, ba thư viện quan trọng được sử dụng là `TRL`, `PEFT` và `Unsloth`. Mỗi thư viện đảm nhiệm một vai trò riêng nhưng có thể kết hợp rất tốt với nhau trong cùng một quy trình.

Trước hết, `TRL` thường được dùng để tổ chức quá trình huấn luyện ở mức cao hơn, đặc biệt thuận tiện cho các bài toán SFT. Thư viện này cung cấp các công cụ giúp ta thiết lập trainer, chuẩn bị dữ liệu theo định dạng hội thoại hoặc instruction-response, và quản lý quá trình huấn luyện một cách dễ theo dõi hơn.

Tiếp theo, `PEFT` là thành phần trung tâm khi làm việc với các kỹ thuật fine-tuning hiệu quả tham số. Thư viện này cho phép gắn adapter vào mô hình, cấu hình LoRA hoặc QLoRA, và quản lý phần tham số cần huấn luyện mà không phải tự can thiệp quá sâu vào kiến trúc bên trong của mô hình.

Cuối cùng, `Unsloth` được sử dụng để tối ưu tốc độ và bộ nhớ trong quá trình huấn luyện. Điều này đặc biệt hữu ích khi làm việc với LLM trên phần cứng giới hạn, vì nó giúp pipeline gọn hơn và thực nghiệm trở nên dễ tiếp cận hơn đối với người học.

Có thể hình dung vai trò của ba thư viện theo chuỗi công việc đơn giản sau:

1. Nạp mô hình nền và tokenizer.
2. Gắn cấu hình LoRA hoặc QLoRA bằng `PEFT`.
3. Chuẩn bị dữ liệu SFT và trainer bằng `TRL`.
4. Tối ưu huấn luyện bằng `Unsloth`.
5. Tiến hành train và kiểm tra phản hồi của mô hình sau fine-tuning.

Sự kết hợp này giúp pipeline fine-tuning trở nên vừa có tính học thuật, vừa có tính thực hành cao. Người học không chỉ hiểu về SFT, LoRA và QLoRA ở mức ý tưởng, mà còn thấy rõ từng công cụ cụ thể đang hỗ trợ phần nào trong quá trình triển khai thực tế.

VII. Thực hành SFT với Llama-3.2-1B-Instruct bằng TRL

Trong phần này, chúng ta thực hành bài toán *Supervised Fine-Tuning* cho một mô hình ngôn ngữ lớn theo định dạng hội thoại. Mục tiêu là tinh chỉnh mô hình `Llama-3.2-1B-Instruct` trên tập dữ liệu `Vi-Alpaca-Preference`, sau đó so sánh hai chế độ huấn luyện hiệu quả tham số là `LoRA` và `QLoRA`. Bài thực hành sử dụng `Transformers`, `TRL`, `PEFT` và `bitsandbytes` để xây dựng một pipeline fine-tuning gọn, rõ ràng và phù hợp cho thực nghiệm.

Bước 1: Cài đặt thư viện và cấu hình chung

```
HF_TOKEN_INPUT = "hf_..."  # hoặc để trống nếu đã login từ trước

!pip install -q -U accelerate peft trl bitsandbytes sentencepiece

import gc, json, os, time
from copy import deepcopy

import torch
from datasets import load_dataset
from peft import LoraConfig, get_peft_model, prepare_model_for_kbit_training
from transformers import AutoModelForCausalLM, AutoTokenizer, BitsAndBytesConfig
from trl import SFTConfig, SFTTrainer

HF_TOKEN = (os.environ.get("HF_TOKEN") or "").strip() or (
    HF_TOKEN_INPUT.strip() if HF_TOKEN_INPUT else None
)
HF_TOKEN = HF_TOKEN or None

MODEL_ID = "meta-llama/Llama-3.2-1B-Instruct"
CACHE_DIR = "./cache"
OUTPUT_ROOT = "./out_full_trl"
os.makedirs(OUTPUT_ROOT, exist_ok=True)

MAX_SEQ_LENGTH = 128
SEED = 42

LORA_R = 16
LORA_ALPHA = 16
LORA_DROPOUT = 0.05
TARGET_MODULES = [
    "q_proj", "k_proj", "v_proj", "o_proj",
    "gate_proj", "up_proj", "down_proj"
```

```
]

USE_BF16 = torch.cuda.is_available() and torch.cuda.is_bf16_supported()
DTYPE = torch.bfloat16 if USE_BF16 else torch.float16

COMMON_TRAIN_KWARGS = dict(
    num_train_epochs=1,
    max_steps=200,
    per_device_train_batch_size=4,
    per_device_eval_batch_size=4,
    gradient_accumulation_steps=8,
    learning_rate=2e-4,
    warmup_steps=10,
    logging_steps=10,
    eval_strategy="steps",
    eval_steps=100,
    save_strategy="no",
    bf16=USE_BF16,
    fp16=not USE_BF16,
    gradient_checkpointing=True,
    optim="adamw_torch",
    report_to="none",
    disable_tqdm=False,
    load_best_model_at_end=False,
    seed=SEED,
)

histories = {}
run_stats = {}
infer_results = {}
```

Khởi đầu quy trình, ta cài đặt các thư viện cần thiết và định nghĩa toàn bộ cấu hình dùng chung cho thí nghiệm. Các siêu tham số như độ dài chuỗi tối đa, hạng của LoRA, learning rate, batch size, số bước huấn luyện và kiểu precision đều được thiết lập ngay từ đầu để giúp hai cấu hình `LoRA` và `QLoRA` có thể dùng chung một khung train thống nhất.

Bước 2: Tải dữ liệu và chuyển sang định dạng hội thoại

```
raw = load_dataset("thainq107/Vi-Alpaca-Preference")

def row_to_messages(example):
    return {
        "messages": [
            {"role": "system",    "content": "You are a helpful assistant."},
            {"role": "user",      "content": example["question"]},
            {"role": "assistant", "content": example["chosen"]},
        ]
    }

```

```python
train_ds = raw["train"].map(
    row_to_messages,
    remove_columns=raw["train"].column_names
)
eval_ds = raw["test"].map(
    row_to_messages,
    remove_columns=raw["test"].column_names
)

TRAIN_SUBSAMPLE = 4000
EVAL_SUBSAMPLE = 800

if TRAIN_SUBSAMPLE:
    train_ds = train_ds.shuffle(SEED).select(
        range(min(TRAIN_SUBSAMPLE, len(train_ds)))
    )
if EVAL_SUBSAMPLE:
    eval_ds = eval_ds.shuffle(SEED).select(
        range(min(EVAL_SUBSAMPLE, len(eval_ds)))
    )

INFER_SAMPLES = [
    {"question": eval_ds[i]["messages"][1]["content"]}
    for i in range(3)
]

print("train:", len(train_ds), "| eval:", len(eval_ds))
```

Tập dữ liệu gốc được chuyển sang định dạng `messages`, gồm ba vai trò quen thuộc là `system`, `user` và `assistant`. Cách tổ chức này rất phù hợp với các mô hình instruction-tuned vì nó phản ánh đúng cấu trúc hội thoại mà mô hình sẽ gặp trong lúc huấn luyện và suy luận. Sau đó, dữ liệu được lấy mẫu con để giúp quá trình thực nghiệm diễn ra nhanh và dễ kiểm soát hơn.

Bước 3: Chuẩn bị các hàm tiện ích cho huấn luyện

```python
def clear_gpu():
    gc.collect()
    torch.cuda.empty_cache()

def apply_chat(example, tok):
    msgs = example["messages"]
    if isinstance(msgs[0], dict):
        return [tok.apply_chat_template(
            msgs, tokenize=False, add_generation_prompt=False
        )]
    return [tok.apply_chat_template(
        m, tokenize=False, add_generation_prompt=False
    ) for m in msgs]

```

```python
def param_stats(model):
    total = sum(p.numel() for p in model.parameters())
    trainable = sum(p.numel() for p in model.parameters() if p.requires_grad)
    return {
        "total": total,
        "trainable": trainable,
        "ratio_pct": 100 * trainable / total
    }

def save_run(output_dir, label, framework, mode, seconds, stats):
    os.makedirs(output_dir, exist_ok=True)
    stat = {
        "run": label,
        "framework": framework,
        "mode": mode,
        "train_seconds": seconds,
        **stats,
    }
    run_stats[label] = stat
    with open(os.path.join(output_dir, "run_stats.json"), "w") as f:
        json.dump(stat, f, indent=2)

def extract_answer(full_text):
    lower = full_text.lower()
    idx = lower.rfind("assistant")
    if idx != -1:
        return full_text[idx + len("assistant"):].strip().lstrip(":\n ")
    return full_text.strip()
```

Các hàm tiện ích trong bước này giúp pipeline gọn hơn và dễ theo dõi hơn. Cụ thể, ta cần một hàm giải phóng bộ nhớ GPU, một hàm biến danh sách hội thoại thành chuỗi prompt theo chat template của tokenizer, một hàm thống kê số tham số có thể huấn luyện, một hàm lưu lại thông tin của từng lần chạy, và một hàm tách riêng phần câu trả lời sau khi sinh văn bản.

Bước 4: Xây dựng hàm huấn luyện TRL cho LoRA/QLoRA

Code Exercise

```python
def train_trl(qlora: bool, output_dir: str, label: str):
    clear_gpu()
    print(f"\n=== {label} ===")

    tok = ***Your Code Here***(
        MODEL_ID,
        cache_dir=CACHE_DIR,
        trust_remote_code=True,
        token=HF_TOKEN,
    )
    if tok.pad_token is None:
        tok.pad_token = tok.eos_token
```

```
    tok.padding_side = "right"

    if qlora:
        bnb_cfg = ***Your Code Here***(
            load_in_4bit=True,
            bnb_4bit_quant_type="nf4",
            bnb_4bit_compute_dtype=DTYPE,
            bnb_4bit_use_double_quant=True,
        )
        model = ***Your Code Here***(
            MODEL_ID,
            quantization_config=bnb_cfg,
            device_map="auto",
            cache_dir=CACHE_DIR,
            trust_remote_code=True,
            token=HF_TOKEN,
        )
        model = ***Your Code Here***(
            model,
            use_gradient_checkpointing=True,
            gradient_checkpointing_kwargs={"use_reentrant": False},
        )
    else:
        model = ***Your Code Here***(
            MODEL_ID,
            torch_dtype=DTYPE,
            device_map="auto",
            cache_dir=CACHE_DIR,
            trust_remote_code=True,
            token=HF_TOKEN,
        )

    model.config.use_cache = False
    model = ***Your Code Here***(
        model,
        LoraConfig(
            r=LORA_R,
            lora_alpha=LORA_ALPHA,
            lora_dropout=LORA_DROPOUT,
            bias="none",
            task_type="CAUSAL_LM",
            target_modules=TARGET_MODULES,
        )
    )

    ps = param_stats(model)
    print(f"trainable={ps['trainable']:,} ({ps['ratio_pct']:.3f}%)")

    kw = dict(COMMON_TRAIN_KWARGS)
    kw["optim"] = "paged_adamw_8bit" if qlora else "adamw_torch"

    args = SFTConfig(
        output_dir=output_dir,
        max_length=MAX_SEQ_LENGTH,
```

```
        dataset_text_field=None,
        **kw
    )

    trainer = ***Your Code Here***(
        model=model,
        processing_class=tok,
        args=args,
        train_dataset=train_ds,
        eval_dataset=eval_ds,
        formatting_func=lambda ex: apply_chat(ex, tok),
    )

    t0 = time.perf_counter()
    trainer.train()
    elapsed = time.perf_counter() - t0
    print(f"[Done] {elapsed:.1f}s")

    adapter_dir = os.path.join(output_dir, "adapter")
    model.save_pretrained(adapter_dir)
    tok.save_pretrained(adapter_dir)

    log = deepcopy(trainer.state.log_history)
    with open(os.path.join(output_dir, "log_history.json"), "w") as f:
        json.dump(log, f, indent=2)

    histories[label] = log
    save_run(
        output_dir, label, "trl",
        "qlora" if qlora else "lora",
        elapsed, ps
    )

    del trainer, model
    clear_gpu()
```

Đây là phần bài tập chính của notebook. Bạn cần hoàn thiện các vị trí còn thiếu theo đúng vai trò của từng bước trong pipeline:

- Vị trí thứ nhất dùng để nạp tokenizer từ checkpoint của mô hình.
- Cụm vị trí trong nhánh `qlora=True` dùng để thiết lập cơ chế lượng tử hóa 4-bit, nạp mô hình nền theo cấu hình đó, rồi chuẩn bị mô hình cho kiểu huấn luyện tiết kiệm bộ nhớ.
- Vị trí ở nhánh còn lại dùng để nạp mô hình trong chế độ thông thường cho bài toán `LoRA`.
- Vị trí tiếp theo dùng để gắn phần adapter vào mô hình gốc.
- Vị trí cuối cùng dùng để tạo bộ huấn luyện SFT từ mô hình, tokenizer, dữ liệu train/eval và hàm định dạng hội thoại.

Điểm quan trọng ở đây là bạn chỉ cần hoàn thiện các thành phần khởi tạo cốt lõi, còn phần khung huấn luyện, lưu adapter, lưu log và thống kê số tham số đã được chuẩn bị sẵn. Nhờ đó, bài tập tập trung đúng vào bản chất của việc ghép `TRL` với `LoRA/QLoRA`, thay vì dàn trải sang quá nhiều chi tiết phụ.

Bước 5: Chạy hai cấu hình huấn luyện và lưu adapter

```
train_trl(
    qlora=False,
    output_dir=os.path.join(OUTPUT_ROOT, "trl_lora"),
    label="TRL + LoRA",
)

train_trl(
    qlora=True,
    output_dir=os.path.join(OUTPUT_ROOT, "trl_qlora"),
    label="TRL + QLoRA",
)
```

Sau khi hoàn thiện hàm huấn luyện, ta gọi hai lần với cùng một pipeline nhưng khác chế độ mô hình. Lần chạy thứ nhất tương ứng với `LoRA`; lần chạy thứ hai dùng `QLoRA`. Mỗi lần chạy sẽ tạo ra một thư mục riêng chứa adapter đã huấn luyện, tokenizer tương ứng, lịch sử loss và các thống kê cơ bản của phiên train.

Bước 6: Suy luận từ adapter sau khi huấn luyện

```
@torch.inference_mode()
def infer_trl(adapter_dir: str, qlora: bool, max_new_tokens: int = 256):
    clear_gpu()

    tok = AutoTokenizer.from_pretrained(
        adapter_dir, trust_remote_code=True, token=HF_TOKEN
    )
    if tok.pad_token is None:
        tok.pad_token = tok.eos_token

    if qlora:
        bnb = BitsAndBytesConfig(
            load_in_4bit=True,
            bnb_4bit_quant_type="nf4",
            bnb_4bit_compute_dtype=DTYPE,
            bnb_4bit_use_double_quant=True,
        )
        base = AutoModelForCausalLM.from_pretrained(
            MODEL_ID,
            quantization_config=bnb,
            device_map="auto",
```

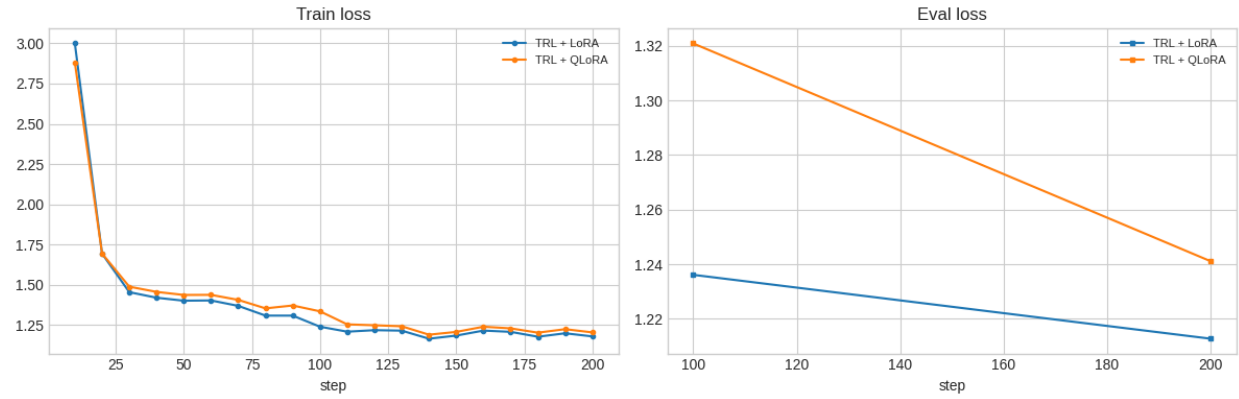
```
            trust_remote_code=True,
            token=HF_TOKEN,
        )
    else:
        base = AutoModelForCausalLM.from_pretrained(
            MODEL_ID,
            torch_dtype=DTYPE,
            device_map="auto",
            trust_remote_code=True,
            token=HF_TOKEN,
        )

    from peft import PeftModel
    model = PeftModel.from_pretrained(base, adapter_dir)
    model.eval()
    model.config.use_cache = True
    device = next(model.parameters()).device

    outs = []
    for item in INFER_SAMPLES:
        msgs = [
            {"role": "system", "content": "You are a helpful assistant."},
            {"role": "user",   "content": item["question"]},
        ]
        prompt = tok.apply_chat_template(
            msgs, tokenize=False, add_generation_prompt=True
        )
        inp = tok(prompt, return_tensors="pt").to(device)
        out = model.generate(
            **inp,
            max_new_tokens=max_new_tokens,
            do_sample=False,
            pad_token_id=tok.eos_token_id,
        )
        outs.append(
            extract_answer(tok.decode(out[0], skip_special_tokens=True))
        )

    del model, base
    clear_gpu()
    return outs
```

Sau giai đoạn huấn luyện, ta nạp lại mô hình nền và adapter để thực hiện suy luận trên một vài mẫu câu hỏi lấy từ tập đánh giá. Bước này cho phép kiểm tra nhanh xem adapter đã học được cách phản hồi theo dữ liệu huấn luyện hay chưa, đồng thời giúp so sánh đầu ra của hai cấu hình `LoRA` và `QLoRA` trong cùng một quy trình sinh văn bản.

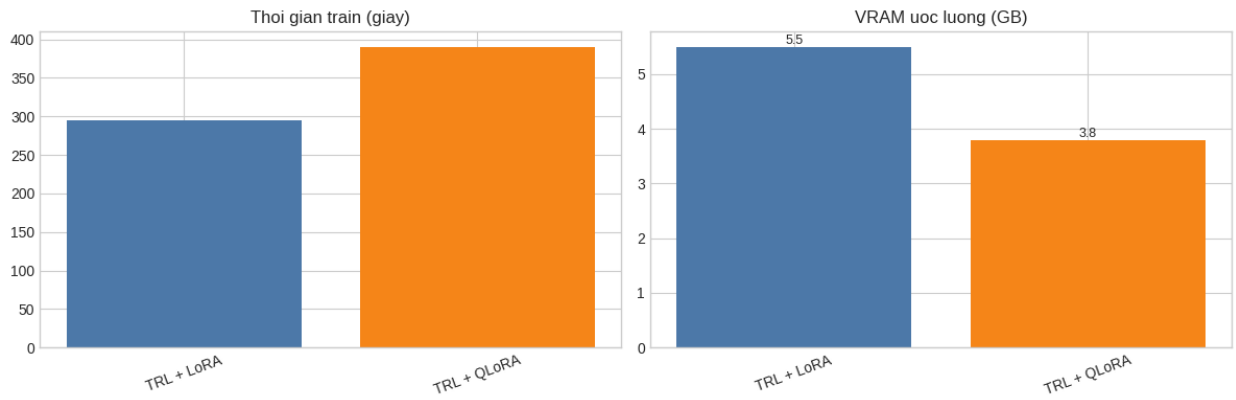
```
try:
    infer_results["TRL + LoRA"] = infer_trl(
        os.path.join(OUTPUT_ROOT, "trl_lora", "adapter"),
        qlora=False
```

```
    )
    print("[OK] TRL + LoRA")
except Exception as e:
    infer_results["TRL + LoRA"] = None
    print(f"[FAIL] TRL + LoRA\n{e}")

try:
    infer_results["TRL + QLoRA"] = infer_trl(
        os.path.join(OUTPUT_ROOT, "trl_qlora", "adapter"),
        qlora=True
    )
    print("[OK] TRL + QLoRA")
except Exception as e:
    infer_results["TRL + QLoRA"] = None
    print(f"[FAIL] TRL + QLoRA\n{e}")
```

Nếu cần mở rộng phần cuối của bài thực hành, bạn có thể bổ sung thêm bước tổng hợp `log_history.json`, `run_stats.json` và các phản hồi suy luận để trực quan hóa train loss, eval loss, thời gian huấn luyện và mức sử dụng bộ nhớ ước lượng. Tuy nhiên, trong phần trình bày chính của tài liệu, các đoạn code trên đã là những thành phần cốt lõi nhất để người học nắm được toàn bộ pipeline SFT bằng `TRL`.



Hình 4: So sánh `train loss` và `eval loss` giữa hai cấu hình `TRL + LoRA` và `TRL + QLoRA`.



Hình 5: So sánh thời gian huấn luyện và mức sử dụng VRAM ước lượng giữa `TRL + LoRA` và `TRL + QLoRA`.

	#	prompt	TRL + LoRA	TRL + QLoRA
0	1	Xu hướng gần đây là gì?	Xu hướng gần đây là sự phát triển của công nghệ và đổi mới trong nhiều lĩnh vực, bao gồm trí tuệ nhân tạo, blockchain, AI, Internet of Things (IoT), và nhiều lĩnh vực khác.	Xu hướng gần đây là sự phát triển của công nghệ và công nghệ thông tin, đặc biệt là trong lĩnh vực AI, blockchain, IoT, và các công nghệ mới như 5G, 6G, và 7G.
1	2	Đưa ra một số dữ liệu ví dụ, phân loại dữ liệu thành các cụm.\n\n[{name: "John"}, {name: "Sara"}, {location: "New York"}, ...	Cụm dữ liệu này có thể được phân loại thành các cụm sau:\n\n- {name: "John", location: "New York"}\n- {name: "Sara", location: "Washington DC"}\n- {name: "John", location: "New York"}\n- {name: "S...	Dữ liệu này có thể được phân loại thành các cụm sau:\n\n- {name: "John"}: danh tính của một người\n- {location: "New York"}: địa điểm của một người\n- {location: "Washington DC"}: địa điểm của một...
2	3	Đưa ra một vật dụng hàng ngày, xây dựng một phép ẩn dụ về nó.\n\ncửa	Một chiếc cửa là một vật dụng hàng ngày rất quan trọng trong cuộc sống của chúng ta. Nó giúp chúng ta tiếp cận với thế giới bên ngoài, cho phép chúng ta tiếp xúc với những trải nghiệm mới, khám ph...	Một chiếc cửa là một cánh cửa để mở ra một cánh cửa khác.

Hình 6: So sánh đầu ra suy luận của hai mô hình `TRL + LoRA` và `TRL + QLoRA` trên một số prompt đánh giá.

VIII. Thực hành SFT với Llama-3.2-1B-Instruct bằng Unsloth

Trong phần này, chúng ta tiếp tục bài toán *Supervised Fine-Tuning* cho mô hình ngôn ngữ lớn, nhưng thay vì đi theo pipeline `Transformers + TRL` thuần túy, ta sử dụng `Unsloth` để tối ưu tốc độ và mức sử dụng bộ nhớ trong quá trình huấn luyện. Mục tiêu là tinh chỉnh mô hình `Llama-3.2-1B-Instruct` trên tập dữ liệu `Vi-Alpaca-Preference`, sau đó so sánh hai chế độ `LoRA` và `QLoRA` trong cùng một khung thực nghiệm.

Bước 1: Cài đặt thư viện và cấu hình chung

```
!pip install -q -U accelerate peft trl bitsandbytes sentencepiece
!pip install -q unsloth

import unsloth  # phải import trước trl/transformers/peft
import gc, json, os, time
from copy import deepcopy

import torch
from datasets import load_dataset
from trl import SFTConfig, SFTTrainer
from unsloth import FastLanguageModel

MODEL_ID = "unsloth/Llama-3.2-1B-Instruct"
CACHE_DIR = "./cache"
OUTPUT_ROOT = "./out_full_unsloth"
os.makedirs(OUTPUT_ROOT, exist_ok=True)

MAX_SEQ_LENGTH = 128
SEED = 42

LORA_R = 16
LORA_ALPHA = 16
LORA_DROPOUT = 0
TARGET_MODULES = [
    "q_proj", "k_proj", "v_proj", "o_proj",
    "gate_proj", "up_proj", "down_proj"
]

USE_BF16 = torch.cuda.is_available() and torch.cuda.is_bf16_supported()
DTYPE = torch.bfloat16 if USE_BF16 else torch.float16

COMMON_TRAIN_KWARGS = dict(
    num_train_epochs=1,
    max_steps=200,
    per_device_train_batch_size=4,
    per_device_eval_batch_size=4,
    gradient_accumulation_steps=8,
```

```
    learning_rate=2e-4,
    warmup_steps=10,
    logging_steps=10,
    eval_strategy="steps",
    eval_steps=100,
    save_strategy="no",
    bf16=USE_BF16,
    fp16=not USE_BF16,
    gradient_checkpointing=True,
    optim="adamw_torch",
    report_to="none",
    disable_tqdm=False,
    load_best_model_at_end=False,
    seed=SEED,
)

histories = {}
run_stats = {}
infer_results = {}
```

Điểm khác biệt đầu tiên của notebook này là mô hình sử dụng checkpoint công khai từ `unsloth`, nên không cần xin quyền truy cập Meta hay khai báo `HF_TOKEN`. Đồng thời, `unsloth` phải được import trước các thư viện liên quan để pipeline hoạt động đúng. Phần còn lại của cấu hình vẫn giữ tinh thần giống bài trước: cố định độ dài chuỗi, thiết lập các siêu tham số LoRA, và chuẩn bị sẵn một bộ đối số huấn luyện dùng chung cho cả hai chế độ.

Bước 2: Tải dữ liệu và chuẩn hóa sang định dạng hội thoại

```
raw = load_dataset("thainq107/Vi-Alpaca-Preference")

def row_to_messages(example):
    return {
        "messages": [
            {"role": "system",    "content": "You are a helpful assistant."},
            {"role": "user",      "content": example["question"]},
            {"role": "assistant", "content": example["chosen"]},
        ]
    }

train_ds = raw["train"].map(
    row_to_messages,
    remove_columns=raw["train"].column_names
)
eval_ds = raw["test"].map(
    row_to_messages,
    remove_columns=raw["test"].column_names
)

TRAIN_SUBSAMPLE = 4000
```

```python
EVAL_SUBSAMPLE = 800

if TRAIN_SUBSAMPLE:
    train_ds = train_ds.shuffle(SEED).select(
        range(min(TRAIN_SUBSAMPLE, len(train_ds)))
    )
if EVAL_SUBSAMPLE:
    eval_ds = eval_ds.shuffle(SEED).select(
        range(min(EVAL_SUBSAMPLE, len(eval_ds)))
    )

INFER_SAMPLES = [
    {"question": eval_ds[i]["messages"][1]["content"]}
    for i in range(3)
]

print("train:", len(train_ds), "| eval:", len(eval_ds))
```

Tập dữ liệu vẫn được chuyển về dạng hội thoại ba vai trò `system`, `user`, `assistant`. Cấu trúc này giúp mô hình học đúng định dạng prompt–response trong bối cảnh instruction tuning. Sau đó, dữ liệu train và eval được lấy mẫu con để giảm thời gian thực nghiệm, đồng thời trích sẵn một vài câu hỏi cho phần suy luận sau khi huấn luyện.

Bước 3: Các hàm tiện ích phục vụ huấn luyện và suy luận

```python
def clear_gpu():
    gc.collect()
    torch.cuda.empty_cache()

def apply_chat(example, tok):
    msgs = example["messages"]
    if isinstance(msgs[0], dict):
        return [tok.apply_chat_template(
            msgs, tokenize=False, add_generation_prompt=False
        )]
    return [tok.apply_chat_template(
        m, tokenize=False, add_generation_prompt=False
    ) for m in msgs]

def param_stats(model):
    total = sum(p.numel() for p in model.parameters())
    trainable = sum(p.numel() for p in model.parameters() if p.requires_grad)
    return {
        "total": total,
        "trainable": trainable,
        "ratio_pct": 100 * trainable / total
    }

def save_run(output_dir, label, framework, mode, seconds, stats):
```

```
    os.makedirs(output_dir, exist_ok=True)
    stat = {
        "run": label,
        "framework": framework,
        "mode": mode,
        "train_seconds": seconds,
        **stats,
    }
    run_stats[label] = stat
    with open(os.path.join(output_dir, "run_stats.json"), "w") as f:
        json.dump(stat, f, indent=2)

def extract_answer(full_text):
    lower = full_text.lower()
    idx = lower.rfind("assistant")
    if idx != -1:
        return full_text[idx + len("assistant"):].strip().lstrip(":\n ")
    return full_text.strip()
```

Các hàm tiện ích ở đây có vai trò giống phiên bản trước: giải phóng bộ nhớ GPU, áp dụng chat template để tạo chuỗi huấn luyện, thống kê tỷ lệ tham số được cập nhật, lưu thông tin của từng phiên chạy, và tách phần nội dung trả lời từ chuỗi sinh ra sau suy luận.

Bước 4: Xây dựng hàm huấn luyện với `Unsloth`

Code Exercise

```
def train_unsloth(load_in_4bit: bool, output_dir: str, label: str):
    clear_gpu()
    print(f"\n=== {label} ===")

    model, tok = ***Your Code Here***(
        model_name=MODEL_ID,
        max_seq_length=MAX_SEQ_LENGTH,
        dtype=DTYPE,
        load_in_4bit=load_in_4bit,
        cache_dir=CACHE_DIR,
    )
    if tok.pad_token is None:
        tok.pad_token = tok.eos_token
    tok.padding_side = "right"

    model = ***Your Code Here***(
        model,
        r=LORA_R,
        target_modules=TARGET_MODULES,
        lora_alpha=LORA_ALPHA,
        lora_dropout=LORA_DROPOUT,
        bias="none",
        use_gradient_checkpointing="unsloth",
```

```
        random_state=SEED,
    )

    ps = param_stats(model)
    print(f"trainable={ps['trainable']:,} ({ps['ratio_pct']:.3f}%)")

    kw = dict(COMMON_TRAIN_KWARGS)
    kw["optim"] = "paged_adamw_8bit" if load_in_4bit else "adamw_torch"

    args = SFTConfig(
        output_dir=output_dir,
        max_length=MAX_SEQ_LENGTH,
        dataset_text_field=None,
        **kw
    )

    trainer = ***Your Code Here***(
        model=model,
        processing_class=tok,
        args=args,
        train_dataset=train_ds,
        eval_dataset=eval_ds,
        formatting_func=lambda ex: apply_chat(ex, tok),
    )

    t0 = time.perf_counter()
    trainer.train()
    elapsed = time.perf_counter() - t0
    print(f"[Done] {elapsed:.1f}s")

    adapter_dir = os.path.join(output_dir, "adapter")
    model.save_pretrained(adapter_dir)
    tok.save_pretrained(adapter_dir)

    log = deepcopy(trainer.state.log_history)
    with open(os.path.join(output_dir, "log_history.json"), "w") as f:
        json.dump(log, f, indent=2)

    histories[label] = log
    save_run(
        output_dir, label, "unsloth",
        "qlora" if load_in_4bit else "lora",
        elapsed, ps
    )

    del trainer, model
    clear_gpu()
```

Đây là phần bài tập chính của notebook Unsloth. Các chỗ trống cần được hoàn thiện theo đúng vai trò sau:

- Vị trí đầu tiên dùng để nạp mô hình và tokenizer theo API tốc độ cao của `Unsloth`, đồng thời cho phép chọn có hay không chế độ 4-bit.

- Vị trí thứ hai dùng để gắn adapter vào mô hình nền, tức là biến mô hình hiện tại thành mô hình có thể fine-tune theo cơ chế LoRA.
- Vị trí cuối cùng dùng để tạo bộ huấn luyện SFT, nơi mô hình, tokenizer, dữ liệu và hàm định dạng hội thoại được kết nối lại trong một pipeline hoàn chỉnh.

Ý tưởng quan trọng của bài này là người học chỉ cần tập trung vào ba thao tác cốt lõi nhất khi làm việc với `Unsloth`: nạp mô hình, gắn adapter, và khởi tạo trainer. Phần còn lại như lưu adapter, ghi log và thống kê tham số đã được chuẩn bị sẵn để giúp bài tập ngắn gọn và bám đúng trọng tâm.

Bước 5: Chạy hai cấu hình `LoRA` và `QLoRA`

```
train_unsloth(
    load_in_4bit=False,
    output_dir=os.path.join(OUTPUT_ROOT, "unsloth_lora"),
    label="Unsloth + LoRA",
)

train_unsloth(
    load_in_4bit=True,
    output_dir=os.path.join(OUTPUT_ROOT, "unsloth_qlora"),
    label="Unsloth + QLoRA",
)
```

Sau khi hoàn thiện hàm huấn luyện, ta chạy hai phiên thực nghiệm. Phiên đầu dùng mô hình ở dạng thông thường để huấn luyện với `LoRA`. Phiên sau bật chế độ 4-bit để huấn luyện theo hướng `QLoRA`. Mỗi phiên sẽ lưu lại adapter, tokenizer và log huấn luyện trong một thư mục riêng.

Bước 6: Suy luận với adapter sau khi fine-tuning

```
@torch.inference_mode()
def infer_unsloth(adapter_dir: str, load_in_4bit: bool, max_new_tokens: int = 256):
    clear_gpu()

    model, tok = FastLanguageModel.from_pretrained(
        model_name=adapter_dir,
        max_seq_length=MAX_SEQ_LENGTH,
        dtype=DTYPE,
        load_in_4bit=load_in_4bit,
    )
    FastLanguageModel.for_inference(model)
    model.config.use_cache = True
    device = next(model.parameters()).device

    outs = []
    for item in INFER_SAMPLES:
```

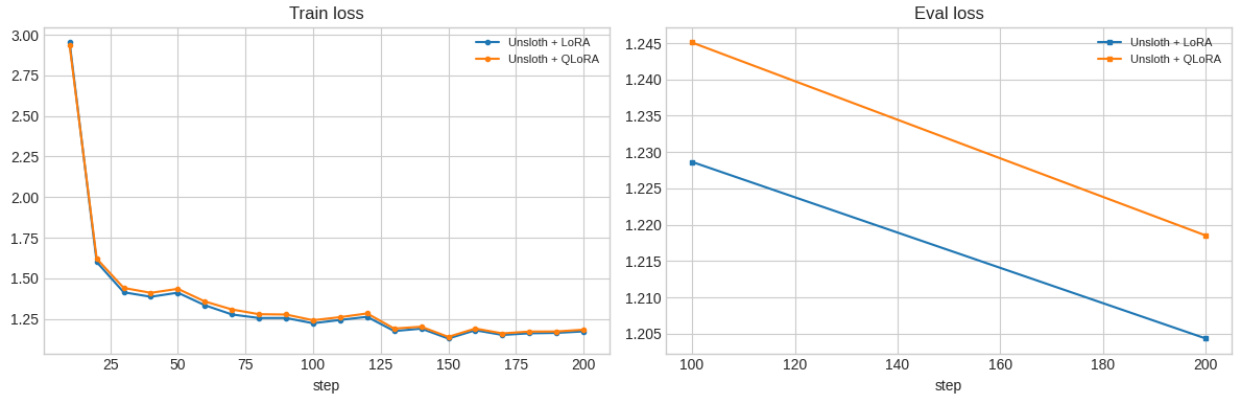
```
        msgs = [
            {"role": "system", "content": "You are a helpful assistant."},
            {"role": "user",   "content": item["question"]},
        ]
        prompt = tok.apply_chat_template(
            msgs, tokenize=False, add_generation_prompt=True
        )
        inp = tok(prompt, return_tensors="pt").to(device)
        out = model.generate(
            **inp,
            max_new_tokens=max_new_tokens,
            do_sample=False,
            pad_token_id=tok.eos_token_id,
        )
        outs.append(
            extract_answer(tok.decode(out[0], skip_special_tokens=True))
        )

    del model
    clear_gpu()
    return outs
```

Khác với pipeline trước, ở đây adapter được nạp lại trực tiếp thông qua cơ chế của `FastLanguageModel`. Sau đó, mô hình được chuyển sang chế độ suy luận để sinh phản hồi trên một số mẫu câu hỏi đánh giá. Cách làm này giúp kiểm tra nhanh chất lượng của mô hình sau fine-tuning mà không cần xây dựng thêm một pipeline riêng cho inference.

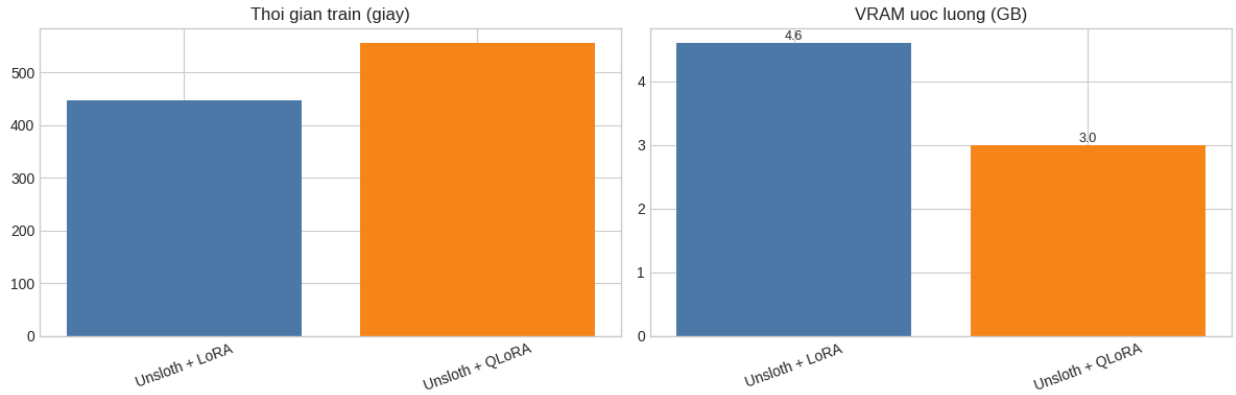
```
try:
    infer_results["Unsloth + LoRA"] = infer_unsloth(
        os.path.join(OUTPUT_ROOT, "unsloth_lora", "adapter"),
        load_in_4bit=False
    )
    print("[OK] Unsloth + LoRA")
except Exception as e:
    infer_results["Unsloth + LoRA"] = None
    print(f"[FAIL] Unsloth + LoRA\n{e}")

try:
    infer_results["Unsloth + QLoRA"] = infer_unsloth(
        os.path.join(OUTPUT_ROOT, "unsloth_qlora", "adapter"),
        load_in_4bit=True
    )
    print("[OK] Unsloth + QLoRA")
except Exception as e:
    infer_results["Unsloth + QLoRA"] = None
    print(f"[FAIL] Unsloth + QLoRA\n{e}")
```

Nếu muốn mở rộng phần phân tích, ta có thể tiếp tục tổng hợp các tệp `log_history.json`, `run_stats.json` và các đầu ra suy luận để trực quan hóa loss, thời gian huấn luyện, VRAM

ước lượng và chất lượng phản hồi. Tuy nhiên, đối với phần nội dung chính của tài liệu, các khối code trên đã đủ để người học nắm được toàn bộ pipeline SFT bằng `Unsloth`.



Hình 7: So sánh `train loss` và `eval loss` giữa hai cấu hình `Unsloth + LoRA` và `Unsloth + QLoRA`.



Hình 8: So sánh thời gian huấn luyện và mức sử dụng VRAM ước lượng giữa `Unsloth + LoRA` và `Unsloth + QLoRA`.

	#	prompt	Unsloth + LoRA	Unsloth + QLoRA
0	1	Xu hướng gần đây là gì?	Xu hướng gần đây là sự phát triển của công nghệ và đổi mới trong lĩnh vực giáo dục, công nghệ thông tin, tài chính và nhiều lĩnh vực khác.	Xu hướng gần đây là sự phát triển của công nghệ và công nghệ thông tin, đặc biệt là trong lĩnh vực AI, blockchain, IoT, và các công nghệ mới như 5G, 6G, và Quantum Computing.
1	2	Đưa ra một số dữ liệu ví dụ, phân loại dữ liệu thành các cụm.\n\n[{name: "John"}, {name: "Sara"}, {location: "New York"}, ...	Cụm dữ liệu này có thể được phân loại thành các cụm sau:\n\n- {name: "John"}\n- {name: "Sara"}\n- {location: "New York"}\n- {location: "Washington DC"}	Cụm dữ liệu này có thể được phân loại thành các nhóm khác nhau như sau:\n\n- Cụm dữ liệu có tên: {name: "John", location: "New York"}\n- Cụm dữ liệu có tên: {name: "Sara", location: "Washington DC...
2	3	Đưa ra một vật dụng hàng ngày, xây dựng một phép ẩn dụ về nó.\n\ncửa	Một chiếc cửa là một vật dụng hàng ngày rất quan trọng trong cuộc sống của chúng ta. Nó giúp chúng ta tiếp cận với thế giới bên ngoài, cho phép chúng ta tiếp cận với những trải nghiệm mới, khám ph...	Một chiếc cửa sổ.

Hình 9: So sánh đầu ra suy luận của hai mô hình `Unsloth + LoRA` và `Unsloth + QLoRA` trên một số prompt đánh giá.

IX. Câu hỏi trắc nghiệm

1. (Tính toán – Tensor flow qua embedding)

Một mô hình nhận đầu vào là tensor `input_ids` có kích thước $[B, S] = [4, 128]$, trong đó B là batch size và S là sequence length. Biết rằng mô hình dùng embedding dimension $d_{\text{model}} = 2048$. Sau khi đi qua **token embedding layer**, tensor đầu ra có kích thước nào?

A. $[4, 2048]$

B. $[4, 128, 2048]$

C. $[128, 4, 2048]$

D. $[4, 128, 128256]$

2. (Tính toán – Kích thước mỗi attention head)

Một mô hình `Llama-3.2-1B` có hidden size $d_{\text{model}} = 2048$ và số attention heads là $h = 32$. Giả sử các head được chia đều, kích thước của mỗi head là bao nhiêu?

A. 16

B. 32

C. 48

D. 64

3. (Tính toán – Số tham số trainable của LoRA cho một linear layer)

Xét một ma trận trọng số gốc $W_0 \in \mathbb{R}^{2048\times 2048}$. Khi áp dụng LoRA với rank $r = 16$, ta có hai ma trận trainable $A \in \mathbb{R}^{2048\times 16}$ và $B \in \mathbb{R}^{16\times 2048}$. Hỏi tổng số tham số **trainable** mới được thêm vào là bao nhiêu?

A. 65,536

B. 131,072

C. 4,194,304

D. 8,388,608

4. (Tính toán – Bộ nhớ của ma trận khi dùng QLoRA)

Vẫn với ma trận $W_0 \in \mathbb{R}^{2048\times 2048}$, số phần tử là $2048 \times 2048 = 4{,}194{,}304$. Nếu lưu ma trận này ở dạng **4-bit** trong QLoRA, và bỏ qua toàn bộ overhead khác, thì bộ nhớ cần dùng xấp xỉ là bao nhiêu?

A. 16 MB

B. 8 MB

C. 2 MB

D. 1 MB

5. (Tính toán – Effective batch size với gradient accumulation)

Một thí nghiệm SFT dùng cấu hình:

- `per_device_train_batch_size` $= 4$,
- `gradient_accumulation_steps` $= 8$,
- số GPU $= 2$.

Khi đó **effective global batch size** là bao nhiêu?

A. 16

B. 32

C. 48

D. 64

6. (Tính toán – Cross-entropy cho một token)

Tại một vị trí sinh token, mô hình cho logits $[2, 0, -1]$ và nhãn đúng là lớp đầu tiên. Xác suất softmax của lớp đúng là

$$p = \frac{e^2}{e^2 + e^0 + e^{-1}}.$$

Cross-entropy loss tại vị trí này là $-\ln p$. Giá trị gần đúng nhất là bao nhiêu?

A. 0.03

B. 0.17

C. 0.51

D. 1.17

7. (Tính toán – Tensor flow của attention scores)

Một tensor hidden states có kích thước $[B, S, d_{\text{model}}] = [2, 128, 2048]$. Mô hình dùng số head $h = 32$ và head dimension $d_{\text{head}} = 64$. Sau khi tạo Q và K, tensor **attention scores** trước softmax, tức là $QK^\top/\sqrt{d_{\text{head}}}$, sẽ có kích thước nào?

A. $[2, 128, 32, 64]$

B. $[2, 128, 128]$

C. $[2, 32, 128, 128]$

D. $[32, 2, 64, 64]$

8. (Tính toán – Tổng số tham số LoRA trên nhiều layer)

Một mô hình có:

- 16 transformer layers,
- mỗi layer gắn LoRA vào 7 target modules,
- mỗi module là một linear layer kích thước 2048×2048,

- rank LoRA $r = 16$.

Biết rằng với mỗi module, số tham số LoRA trainable là $2048 \times 16 + 16 \times 2048 = 65{,}536$. Hỏi tổng số tham số LoRA trainable trên toàn bộ mô hình là bao nhiêu?

A. 7,340,032

B. 14,680,064

C. 67,108,864

D. 117,440,512

9. (Tính toán – Forward pass với công thức LoRA)

Xét công thức LoRA $W = W_0 + AB$. Cho

$$W_0 = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}, \qquad A = \begin{bmatrix} 1 \\ 2 \end{bmatrix}, \qquad B = \begin{bmatrix} 3 & 4 \end{bmatrix}.$$

Gọi x là **vector cột**:

$$x = \begin{bmatrix} 1 \\ 2 \end{bmatrix}.$$

Hỏi đầu ra $y = Wx$ bằng bao nhiêu?

A. $\begin{bmatrix} 11 \\ 23 \end{bmatrix}$

B. $\begin{bmatrix} 12 \\ 24 \end{bmatrix}$

C. $\begin{bmatrix} 13 \\ 26 \end{bmatrix}$

D. $\begin{bmatrix} 14 \\ 28 \end{bmatrix}$

10. (Tính toán – SFT loss trung bình trên nhiều token)

Trong một bài toán SFT, giả sử mô hình cần dự đoán đúng **2 token** liên tiếp. Xác suất mà mô hình gán cho token đúng ở hai vị trí lần lượt là $p_1 = 0.8$ và $p_2 = 0.5$. Loss trung bình trên hai vị trí được tính theo công thức

$$\mathcal{L} = \frac{-\ln p_1 - \ln p_2}{2}.$$

Giá trị gần đúng nhất của loss là bao nhiêu?

A. 0.458

B. 0.693

C. 0.916

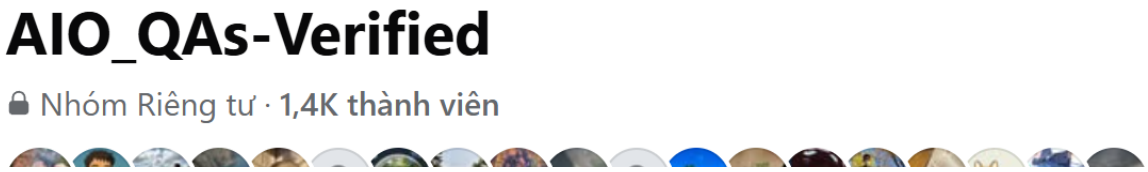
D. 1.204

Phụ lục

1. **Hint:** Các file code gợi ý và dữ liệu nếu có được lưu trong thư mục có thể được tải tại đây.

2. **Solution:** Các file code cài đặt hoàn chỉnh và phần trả lời nội dung trắc nghiệm có thể được tải tại đây (Lưu ý: Sáng thứ 3 khi hết deadline phần project, ad mới copy các nội dung bài giải nêu trên vào đường dẫn).

3. **Q&A:** Bạn có thể đặt thêm câu hỏi về nội dung bài đọc trong group Facebook hỏi đáp tại đây. Tất cả câu hỏi sẽ được trả lời trong vòng tối đa 4 giờ.



Hình 10: Hình ảnh group facebook AIO Q&A

4. **Rubric:**

Phần	Kiến Thức	Đánh Giá
1.	- Hiểu quy trình SFT cho LLM với `Llama-3.2-1B-Instruct` - Nắm được vai trò của `TRL`, `PEFT`, và `Unsloth` - Hiểu cơ chế `LoRA` và `QLoRA` trong fine-tuning hiệu quả tham số	- Hoàn thiện được pipeline huấn luyện với `LoRA` và `QLoRA` - Nạp mô hình, tokenizer, adapter và trainer đúng cách - Lưu được adapter, log huấn luyện và kết quả suy luận
2.	- Hiểu cách so sánh hai cấu hình huấn luyện qua Loss, thời gian train và VRAM - Biết cách suy luận sau fine-tuning và phân tích chất lượng đầu ra - Hiểu mối liên hệ giữa tiết kiệm bộ nhớ và chất lượng mô hình	- So sánh, trực quan hóa được Train/Eval Loss giữa các cấu hình - Đánh giá được thời gian huấn luyện, VRAM ước lượng và phản hồi sinh ra - Rút ra kết luận về ưu, nhược điểm của `LoRA`, `QLoRA` hoặc `Unsloth`